MODULE 1 · LECTURE 8

Computer Programming Basics: Introduction to Visual Basic and Java

How a problem becomes a program — algorithms, flowcharts and the building blocks shared by all languages — followed by a first look at Visual Basic and Java through small farm calculations.

Learning objectives

After this lecture you should be able to:

1. List the steps in developing a program.
2. Write simple algorithms, pseudocode and flowcharts, and use the three basic control structures.
3. Classify programming languages and distinguish compilers from interpreters.
4. Explain the building blocks common to all languages: variables, data types, operators, control statements and procedures.
5. Describe the Visual Basic environment and write a small event-driven program.
6. Explain how Java programs run, its main features and the basic ideas of object-oriented programming.
7. Identify uses of programming in agriculture.

Scope: Lectures 1–7 covered computers, software, databases and the Internet. This lecture introduces how software is written. Excel formulas (Lecture 5) and SQL (Lecture 6) were small steps in this direction. Programs for controllers, models, decision support and artificial intelligence appear in Lectures 11, 12, 16 and 19–28.

1. Programs and programming

A **program** is a set of instructions, written in a programming language, that tells a computer how to carry out a task. **Programming** is the work of designing, writing, testing and maintaining programs, and a **programmer** is the person who does it. A computer does exactly what the program says — nothing more — so the logic has to be complete and precise.

For an agricultural engineer, even a little programming is useful: it automates repeated calculations (fertiliser doses, irrigation schedules, machinery costs), reads data from sensors and controls pumps or motors, analyses field data, and builds simple tools and apps for farmers.

2. Steps in program development

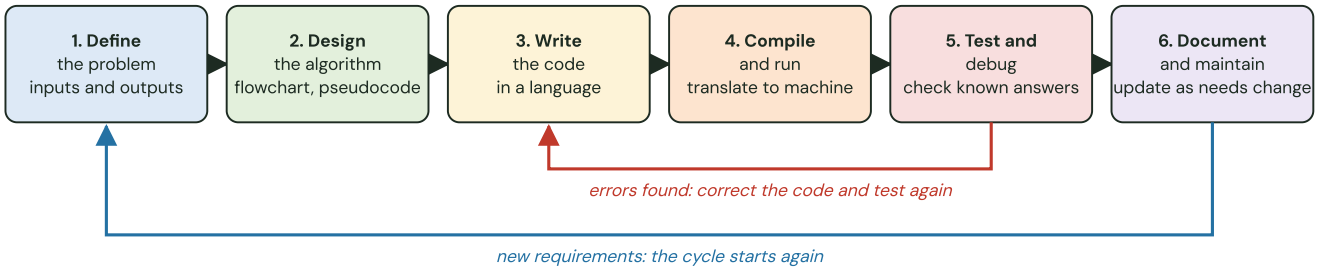


Figure 1. The program development cycle. Original diagram.

1. **Define the problem** — what the program must do, its inputs (e.g. nitrogen dose and area) and its outputs (urea needed).
2. **Design the algorithm** — work out the steps, as a flowchart or pseudocode, before writing any code.
3. **Write the code** — translate the algorithm into a programming language.
4. **Compile and run** — the code is translated into a form the computer can execute (Section 5).
5. **Test and debug** — run the program with inputs whose answers are known and remove errors (**bugs**).
6. **Document and maintain** — explain how the program works and update it when needs change.

3. Algorithms, pseudocode and flowcharts

An **algorithm** is a finite, step-by-step procedure for solving a problem. Knuth (1997) lists five properties of an algorithm:

Property	Meaning
Finiteness	It must end after a finite number of steps
Definiteness	Each step must be precisely and unambiguously stated
Input	It takes zero or more inputs
Output	It produces at least one output
Effectiveness	Each step is simple enough to be carried out exactly, in principle even with pencil and paper

Pseudocode writes an algorithm in structured, plain-language statements that are close to code but not tied to any language. A **flowchart** draws the same algorithm with standard symbols joined by arrows.

Symbol	Shape	Used for
Terminal	Rounded rectangle (oval)	Start and end
Input/output	Parallelogram	Reading data or displaying results
Process	Rectangle	A calculation or action
Decision	Diamond	A yes/no question; two exits
Flow line	Arrow	Order of steps
Connector	Small circle	Joining parts of a chart across a page
Predefined process	Rectangle with double side lines	A sub-procedure defined elsewhere

Worked example — an irrigation controller
Problem: read the soil moisture every 30 minutes; switch the pump on when moisture falls below 25% and off otherwise, until the season ends. (The 25% threshold is illustrative; the right value depends on soil and crop.)

Pseudocode

```
START
LOOP
    READ soil moisture SM (%)
    IF SM < 25 THEN
        switch pump ON
    ELSE
        switch pump OFF
    END IF
    IF season is over THEN EXIT LOOP
    WAIT 30 minutes
END LOOP
STOP
```

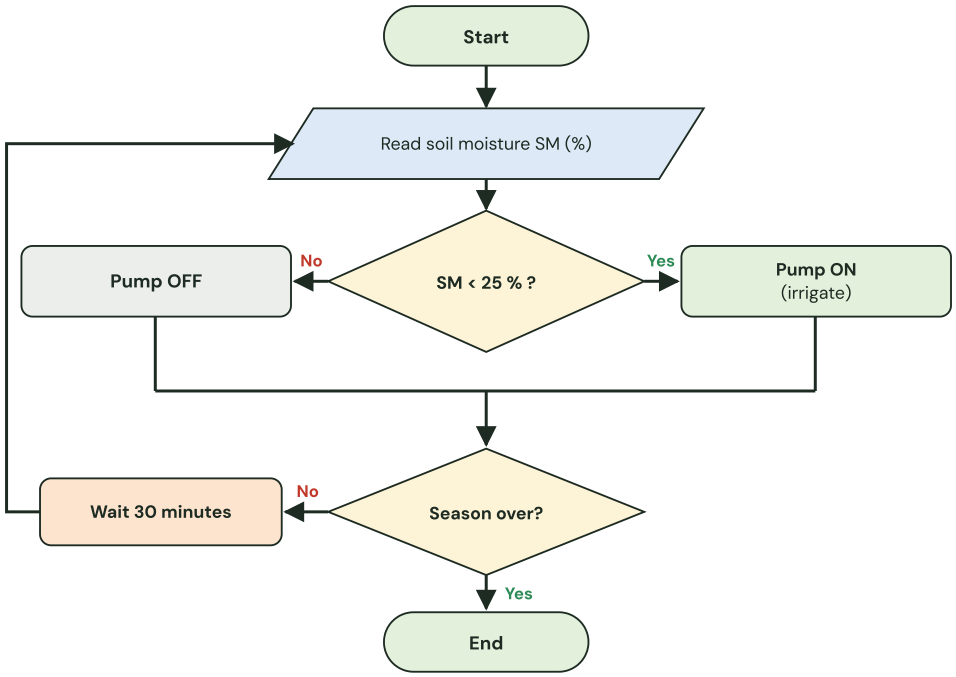


Figure 2. Flowchart of the irrigation controller. Original diagram.

The three control structures

Böhm and Jacopini (1966) showed that any algorithm can be built from just three structures — **sequence**, **selection** and **iteration** — combined and nested. This is the basis of structured programming, and every language provides all three.

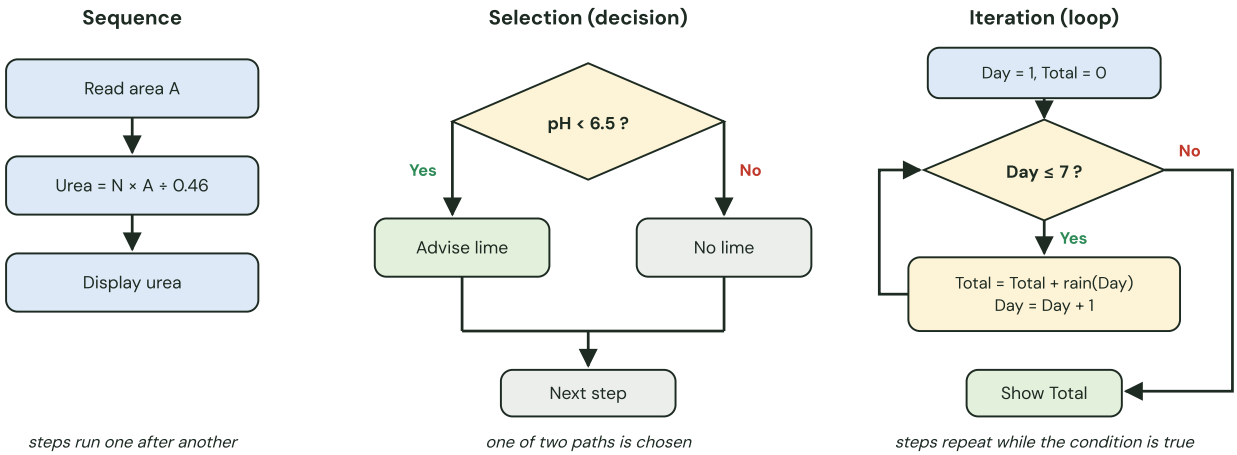


Figure 3. Sequence, selection and iteration, with farm examples. Original diagram.

4. Programming languages

Generation	Type	Description	Examples
1GL	Machine language	Binary instructions executed directly by the processor; fast but very hard to write	10110000 01100001
2GL	Assembly language	Short mnemonics for machine instructions; needs an assembler	MOV, ADD, JMP
3GL	High-level languages	English-like statements, independent of the processor	Fortran, C, BASIC, Visual Basic, Java, Python
4GL	Very high-level / declarative	State what result is wanted rather than how to get it	SQL (Lecture 6), report generators
5GL	Logic and constraint-based	State facts and rules; the system finds a solution	Prolog

Machine and assembly languages are **low-level**: close to the hardware. High-level languages are easier to write and read, and the same program can be moved to other computers. Languages also differ in **style (paradigm)**:

- **Procedural** — the program is a sequence of procedures acting on data (C, Fortran, classic BASIC).
- **Object-oriented** — the program is built from objects that combine data and behaviour (Java, Visual Basic .NET, C#, Python).
- **Event-driven** — the program waits for events such as a button click or a sensor signal and runs the matching code (Visual Basic forms, mobile apps).

5. Translators: assembler, compiler and interpreter

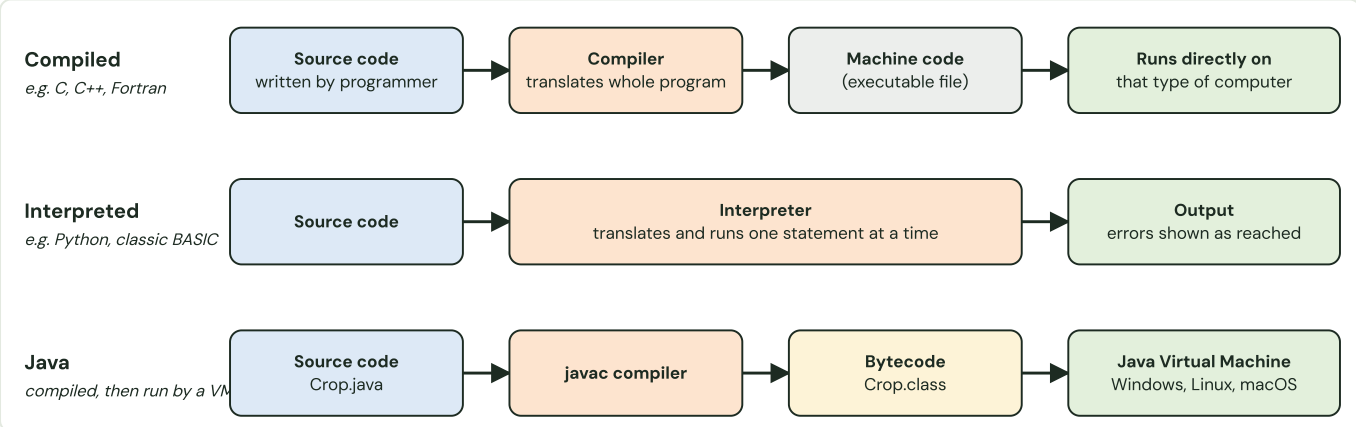


Figure 4. How compiled, interpreted and Java programs are translated and run. Original diagram.

	Compiler	Interpreter
How it works	Translates the whole program into machine code before it runs	Translates and runs the program one statement at a time
Output	An executable file that can be run many times	No separate executable
Speed of the finished program	Faster	Slower
Errors	Reported together after compilation	Reported one at a time as each line is reached
Examples	C, C++, Fortran	Python, classic BASIC

An **assembler** translates assembly language into machine code. Java and Visual Basic .NET use both ideas: they are compiled first into an intermediate code, which a runtime system (the Java Virtual Machine or the .NET runtime) then executes.

6. Building blocks common to all languages

Element	Meaning	Example
Variable	A named storage location whose value can change	area, nRate, totalRain
Constant	A named value that does not change	UREA_N = 0.46
Data type	The kind of value a variable holds	Whole number, decimal number, text, true/false, date
Operator	A symbol that acts on values	Arithmetic + - * /; comparison < > =; logical And, Or, Not
Expression	Values and operators that give a result	nRate * area / 0.46
Input and output	Getting data in and showing results	Reading a text box or sensor; showing a label or printing
Control statements	Sequence, selection (If, Select Case/switch) and iteration (For, While)	Figure 3
Array	A list of values under one name, accessed by position	Seven days of rainfall
Procedure / function / method	A named block of code that can be reused; a function returns a value	SoilReaction(pH)
Comment	A note for people, ignored by the computer	VB: ' text Java: // text

Errors and debugging

Type of error	What happens	Example
Syntax error	The rules of the language are broken; the program will not compile	A missing End If, or a misspelt keyword
Run-time error	The program stops while running	Dividing by an area of zero; text typed where a number is expected
Logic error	The program runs but gives a wrong answer	Using 0.64 instead of 0.46 for the nitrogen content of urea

Debugging is finding and removing errors: test with inputs whose answers you know, check results by hand, and step through the code line by line with the debugger in the development environment.

7. Visual Basic

7.1 About Visual Basic

Visual Basic is an object-oriented programming language developed by Microsoft (Microsoft, n.d.-d). It grew from the BASIC language and is known for **visual** design of the user interface and **event-driven** code. Classic Visual Basic appeared for Windows in 1991 and reached version 6.0 in 1998. The first Visual Basic .NET was released with Visual Studio .NET in 2002 (Microsoft, n.d.-f). Development with Visual Basic 6.0 is no longer supported, and Microsoft recommends moving such programs to newer technology (Microsoft, n.d.-c). Visual Basic .NET remains supported, and Microsoft states that it will keep a stable, approachable design rather than adding many new language features (Microsoft, n.d.-e).

Visual Basic for Applications (VBA) is a version built into Microsoft Office. It is used to write **macros** that automate Excel, Word and other Office programs (Microsoft, n.d.-b) — for example, a macro that formats and prints a soil-test sheet for every village (Lecture 5).

7.2 The development environment

Visual Basic programs are written in **Visual Studio**, an **integrated development environment (IDE)**. To create a Windows Forms app, the programmer drags controls from the **Toolbox** onto a form, sets their properties in the **Properties** window, and double-clicks a control to write the code that runs when it is used (Microsoft, n.d.-a).

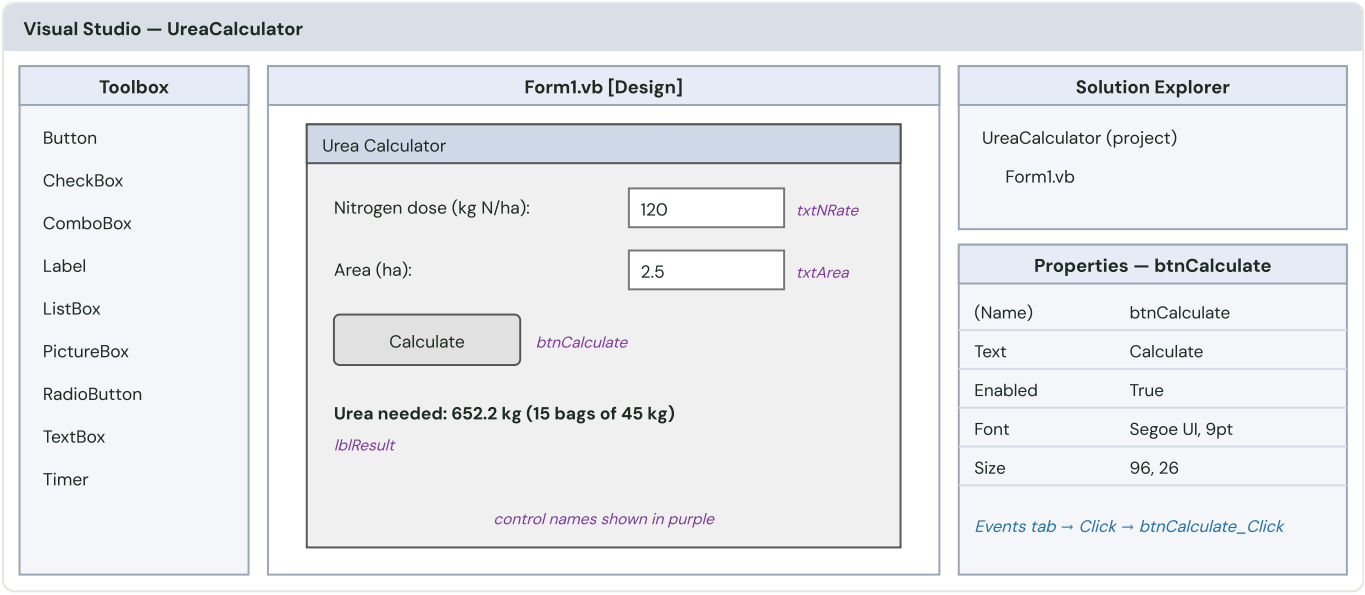


Figure 5. Main parts of the Visual Studio IDE while designing a urea calculator (schematic; layout simplified). Original diagram.

Part of the IDE	Purpose
Form designer	The window being designed — the program's user interface
Toolbox	Controls that can be placed on the form
Properties window	Settings of the selected control: name, text, colour, size, font; its Events tab lists the events it can respond to
Solution Explorer	The files of the project
Code window	Where the event procedures and other code are written
Debugger	Runs the program step by step and shows variable values

7.3 Objects: properties, methods and events

Term	Meaning	Example
Object	Anything the program works with — a form, a button, a text box	btnCalculate
Property	A characteristic of an object	txtArea.Text, btnCalculate.Enabled
Method	An action an object can perform	txtArea.Clear(), Me.Close()
Event	Something that happens to an object, to which the program responds	Click, TextChanged, Load

Common control	Use
Label	Displays text that the user cannot edit — captions and results
TextBox	Lets the user type input — dose, area, date
Button	Starts an action when clicked
CheckBox / RadioButton	Yes/no options / one choice from a group (e.g. irrigated or rainfed)
ComboBox / ListBox	Choose from a list (crop, district)
PictureBox	Shows an image
Timer	Raises an event at fixed intervals — e.g. reading a sensor every minute

7.4 Data types, operators and control statements

Data type	Holds
Integer / Long	Whole numbers (32-bit / 64-bit) — number of bags, days
Single / Double	Decimal numbers — area, yield, rainfall
Decimal	Exact decimal values — money
Boolean	True or False
String / Char	Text / a single character
Date	Date and time

Variables are declared with **Dim**, e.g. Dim area As Double. Arithmetic operators are + - * / ^ (power), \ (integer division) and Mod (remainder); & joins text; comparisons use = <> < > <= >=; logical operators are And, Or and Not.

Worked example 1 — Visual Basic: urea calculator (event procedure behind the Calculate button)

```
Public Class Form1
    ' Runs when the user clicks the Calculate button (an event procedure)
    Private Sub btnCalculate_Click(sender As Object, e As EventArgs) Handles btnCalculate.Click
        Dim nRate As Double = CDbl(txtNRate.Text)        ' kg N per hectare
        Dim area As Double = CDbl(txtArea.Text)          ' hectares
        Const UREA_N As Double = 0.46                    ' urea contains 46% nitrogen
        Dim urea As Double = nRate * area / UREA_N       ' kg of urea
        Dim bags As Integer = CInt(Math.Ceiling(urea / 45))
        lblResult.Text = "Urea needed: " & urea.ToString("F1") & " kg (" & bags & " bags of 45 kg)"
    End Sub
End Class
```

Check by hand: 120 kg N/ha × 2.5 ha ÷ 0.46 = 652.2 kg of urea; 652.2 ÷ 45 = 14.5, rounded up to **15 bags** (assuming 45 kg bags).

Worked example 2 — Visual Basic: Select Case in a function

```
Function SoilReaction(pH As Double) As String
    Select Case pH
        Case Is < 6.5
            Return "Acidic"
        Case 6.5 To 7.5
            Return "Neutral"
        Case Else
            Return "Alkaline"
    End Select
End Function

' SoilReaction(5.8) returns "Acidic"; SoilReaction(7.2) returns "Neutral"; SoilReaction(8.3) returns
"Alkaline"
```

The three pH classes are a simple rating used here only to illustrate Select Case. Both Visual Basic examples compiled without errors with the Microsoft Visual Basic compiler; their results above were checked by hand.

8. Java

8.1 About Java

Java is "a general-purpose, concurrent, class-based, object-oriented language" (Gosling et al., 2025, Chapter 1). It was developed at Sun Microsystems by a team led by James Gosling and released in the mid-1990s; Oracle has developed it since acquiring Sun in 2010. Java is widely used for business and web servers, large data systems and teaching, and it was the main language for Android apps before Google adopted a "Kotlin-first" approach in 2019 (Google, n.d.).

A new version of Java is released every six months. Some versions receive **long-term support (LTS)**: Java SE 8, 11, 17, 21 and 25 are LTS releases, Java 27 was released in September 2026, and the next LTS release, Java 29, is planned for September 2027 (Oracle, n.d.-b).

8.2 How a Java program runs

Java source code is written in files ending in **.java** and compiled by the **javac** compiler into **.class** files. These do not contain machine code for a particular processor but **bytecode** — the machine language of the **Java Virtual Machine (JVM)**. Because JVMs exist for many operating systems, the same .class files can run on Windows, Linux or macOS (Oracle, n.d.-a). This idea is often summed up as "write once, run anywhere" (Figure 4).

Term	Meaning
JVM — Java Virtual Machine	Runs bytecode on a particular computer
JRE — Java Runtime Environment	The JVM plus the standard libraries needed to run Java programs
JDK — Java Development Kit	The JRE plus development tools such as the javac compiler and debugger; needed to write programs

8.3 Main features

- **Object-oriented** — programs are built from classes and objects.
- **Platform-independent** — bytecode runs on any computer with a JVM.
- **Robust and safe** — strong type checking, error handling with exceptions, and no unsafe operations such as array access without index checking (Gosling et al., 2025).

- **Automatic memory management** — a **garbage collector** frees memory that is no longer used (Gosling et al., 2025).
- **Concurrent (multithreaded)** — several tasks can run at the same time, e.g. reading sensors while updating a display.
- **Large standard library** — ready-made classes for text, mathematics, files, networking and user interfaces.

8.4 Writing Java

- Every Java program is made of **classes**; execution starts at the method **public static void main(String[] args)**.
- Java is **case-sensitive**: Area and area are different names. Statements end with a semicolon, and blocks are enclosed in braces { }.
- **Primitive data types:** byte, short, int, long (whole numbers); float, double (decimal numbers); char (a character); boolean (true/false). Text is held in **String** objects.
- **Operators:** + - * / % (remainder); == != < > <= >=; && (and), || (or), ! (not); ++ and -- add or subtract 1.

Worked example 3 — Java: weekly rainfall summary (array, loop and decision)

```
public class RainfallSummary {
    public static void main(String[] args) {
        double[] rain = {0.0, 12.5, 3.0, 0.0, 28.4, 1.2, 6.5};   // mm on 7 days
        double total = 0;
        int rainyDays = 0;
        for (double r : rain) {
            total = total + r;
            if (r >= 2.5) {          // a rainy day: 2.5 mm or more
                rainyDays++;
            }
        }
        double mean = total / rain.length;
        System.out.printf("Total: %.1f mm, mean: %.2f mm/day, rainy days: %d%n", total, mean,
rainyDays);
    }
}

Output:  Total: 51.6 mm, mean: 7.37 mm/day, rainy days: 4
```

8.5 Object-oriented programming (OOP)

Concept	Meaning	Agricultural example
Class	A template that defines data (fields) and behaviour (methods)	class Crop with name, area and yield
Object	A particular instance of a class	wheat and mustard are two Crop objects
Encapsulation	Keeping data and methods together and hiding the data inside the class	Fields marked private; read through getName()
Inheritance	A new class reuses and extends an existing one	class Tractor extends Machine
Polymorphism	One method name behaves differently for different classes	fuelCost() worked out differently for a tractor and a pump
Abstraction	Showing only essential features and hiding detail	A Pump class offers start() and stop() without showing the wiring

Worked example 4 — Java: a class and two objects

```
class Crop {
    private String name;          // fields (data), hidden inside the class
    private double areaHa;
    private double yieldTPerHa;

    Crop(String name, double areaHa, double yieldTPerHa) {   // constructor
        this.name = name;
        this.areaHa = areaHa;
        this.yieldTPerHa = yieldTPerHa;
    }

    double production() {         // method (behaviour)
        return areaHa * yieldTPerHa;
    }

    String getName() { return name; }
}

public class CropDemo {
    public static void main(String[] args) {
        Crop wheat = new Crop("Wheat", 2.5, 4.8);     // two objects of one class
        Crop mustard = new Crop("Mustard", 1.2, 1.6);
        System.out.println(wheat.getName() + ": " + wheat.production() + " t");
        System.out.printf("%s: %.2f t%n", mustard.getName(), mustard.production());
    }
}

Output:  Wheat: 12.0 t
         Mustard: 1.92 t
```

Worked examples 3 and 4 were compiled and run with Java 21; the outputs shown are the actual outputs. The rainfall values are illustrative, and the 2.5 mm rainy-day threshold matches the definition used in Lecture 5.

9. Visual Basic and Java compared

Feature	Visual Basic .NET	Java
Developer	Microsoft	Sun Microsystems; now Oracle
Runs on	The .NET runtime; Windows Forms apps run on Windows	Any computer with a JVM
Case-sensitive?	No	Yes
Declaration	Dim area As Double	double area;
Decision	If x < 25 Then … Else … End If	if (x < 25) { … } else { … }
Multi-way choice	Select Case … End Select	switch (…) { case …: }
Counted loop	For i = 1 To 7 … Next	for (int i = 1; i <= 7; i++) { … }
Conditional loop	Do While x < 10 … Loop	while (x < 10) { … }
Procedure	Sub … End Sub; Function … End Function	Methods, e.g. double production() { … }
Comment	' comment	// comment
Typical uses	Windows desktop tools, Office automation (VBA)	Server and web applications, large systems, Android (older apps)

10. Programming in agriculture

Application	What programs do	Typical languages	Lecture
Farm calculators and decision tools	Fertiliser, seed-rate, irrigation and machinery-cost calculations	Visual Basic, Excel VBA, Java, Python	5, 13, 16
Mobile apps for farmers	Prices, advisories, record keeping	Kotlin and Java (Android)	14
Sensors and controllers	Read sensors, switch pumps and motors (like Figure 2)	C/C++ on microcontrollers	12, 23
Crop and weather models	Simulate crop growth, water balance and weather	Fortran, C++, Python, R	11
Data analysis and statistics	Analyse field trials and yield data	R, Python	11, 26
Databases and web portals	Store and serve farm data online	SQL, Java, JavaScript	6, 7
Artificial intelligence	Image-based pest detection, yield prediction, robotics	Python	19–28

Good programming practice: use meaningful names (urea, not x); write comments and state units (kg/ha, mm); indent blocks; check user input (no negative area); test with values whose answers you know; and keep a copy of each working version.

11. Key takeaways

- A program is a precise set of instructions; developing one means defining the problem, designing an algorithm, coding, translating, testing and maintaining it.
- Algorithms are finite, definite and effective; they are written as pseudocode or drawn as flowcharts.
- Any algorithm can be built from sequence, selection and iteration (Böhm & Jacopini, 1966).
- Languages range from machine and assembly language to high-level, 4GL and 5GL languages; compilers translate a whole program, interpreters one statement at a time.
- Visual Basic is an event-driven, object-oriented Microsoft language: forms are designed visually in Visual Studio, and code runs when events such as Click occur.
- Java source is compiled to bytecode that runs on any JVM; it is object-oriented, robust and has automatic memory management.
- Classes, objects, encapsulation, inheritance, polymorphism and abstraction are the core ideas of object-oriented programming.
- Programming supports agriculture through calculators, apps, controllers, models, data analysis and AI.

12. Examination questions

Short-answer questions (2 marks)

Q1. Define an algorithm. List its properties.

A finite, step-by-step procedure for solving a problem; finiteness, definiteness, input, output and effectiveness.

Q2. What is a flowchart? Give the symbols for a decision and for input/output.

A diagram of an algorithm using standard symbols; decision — diamond; input/output — parallelogram.

Q3. Distinguish between a compiler and an interpreter.

A compiler translates the whole program into machine code before it runs; an interpreter translates and runs one statement at a time.

Q4. What are syntax errors and logic errors?

Syntax errors break the language rules and stop compilation; logic errors let the program run but give wrong results.

Q5. What is event-driven programming?

A style in which code runs in response to events such as a button click, a key press or a timer.

Q6. Name four Visual Basic controls and their uses.

Label (display text), TextBox (input), Button (start an action), ComboBox (choose from a list).

Q7. Write a Visual Basic statement to declare a decimal variable named area.

Dim area As Double

Q8. What is bytecode?

The intermediate code produced by the Java compiler; it is executed by the Java Virtual Machine.

Q9. Expand JDK, JRE and JVM.

Java Development Kit, Java Runtime Environment and Java Virtual Machine.

Q10. Define class and object.

A class is a template defining data and methods; an object is an instance of a class.

Long-answer questions (5–10 marks) — answer outlines

Q1. Explain the steps in developing a program, with a diagram.

Figure 1: define the problem, design the algorithm, code, compile and run, test and debug, document and maintain; error types and debugging.

Q2. Draw a flowchart and write pseudocode for an automatic irrigation controller. Explain the symbols used.

Figure 2 and the pseudocode in Section 3; terminal, input/output, process, decision and flow-line symbols; selection and iteration.

Q3. Describe the generations of programming languages and the types of language translators.

Section 4 table (1GL–5GL); low- and high-level languages; paradigms; assembler, compiler, interpreter (Figure 4 and comparison table).

Q4. Describe the Visual Basic IDE and write a program to calculate the urea needed for a field.

Figure 5; toolbox, properties, Solution Explorer, code window; properties, methods and events; worked example 1 with a hand check.

Q5. Explain the features of Java and how a Java program is executed. Write a program to find the total and mean of a week's rainfall.

Features; .java → javac → .class bytecode → JVM; JDK, JRE, JVM; worked example 3.

Q6. Explain the concepts of object-oriented programming with agricultural examples, and compare Visual Basic with Java.

Class, object, encapsulation, inheritance, polymorphism, abstraction (Section 8.5 and worked example 4); comparison table in Section 9.

13. Glossary

Term	Meaning
Algorithm	A finite, step-by-step procedure for solving a problem
Bug / debugging	An error in a program / finding and removing errors
Bytecode	Intermediate code produced by the Java compiler and run by the JVM
Class	A template for objects, defining their data and methods
Compiler	Translates a whole program into machine code before it runs
Constant	A named value that cannot change while the program runs
Control (VB)	An object placed on a form — label, text box, button
Data type	The kind of value a variable can hold
Event	An action, such as a click, to which a program responds
Flowchart	A diagram of an algorithm drawn with standard symbols
IDE	Integrated development environment — editor, designer, compiler and debugger in one program
Interpreter	Translates and runs a program one statement at a time
JVM	Java Virtual Machine — executes bytecode
Loop	A set of statements repeated while a condition holds
Method	A procedure that belongs to a class or object
Object	An instance of a class, with its own data
Property	A characteristic of an object, such as Text or Enabled
Pseudocode	An algorithm written in structured plain language
Syntax	The grammar rules of a programming language
Variable	A named storage location whose value can change

References

Böhm, C., & Jacopini, G. (1966). Flow diagrams, Turing machines and languages with only two formation rules. *Communications of the ACM, 9*(5), 366–371. https://doi.org/10.1145/355592.365646

Deitel, P., & Deitel, H. (2018). *Java how to program, early objects* (11th ed.). Pearson. ISBN 978-0-13-474335-6

Google. (n.d.). *Android's Kotlin-first approach*. Android Developers. Retrieved November 9, 2026, from https://developer.android.com/kotlin/first

Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D., & Bierman, G. (2025). *The Java language specification: Java SE 25 edition*. Oracle. https://docs.oracle.com/javase/specs/jls/se25/html/index.html

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley. ISBN 978-0-201-89683-1

Microsoft. (n.d.-a). *Tutorial: Create a Windows Forms app with Visual Basic*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/visualstudio/ide/create-a-visual-basic-winform-in-visual-studio

Microsoft. (n.d.-b). *Getting started with VBA in Office*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/office/vba/library-reference/concepts/getting-started-with-vba-in-office

Microsoft. (n.d.-c). *Support statement for Visual Basic 6.0*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/previous-versions/visualstudio/visual-basic-6/visual-basic-6-support-policy

Microsoft. (n.d.-d). *Visual Basic documentation*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/dotnet/visual-basic/

Microsoft. (n.d.-e). *Visual Basic language strategy*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/dotnet/visual-basic/getting-started/strategy

Microsoft. (n.d.-f). *What's new for Visual Basic*. Microsoft Learn. Retrieved November 9, 2026, from https://learn.microsoft.com/en-us/dotnet/visual-basic/getting-started/whats-new

Oracle. (n.d.-a). *About the Java technology*. The Java tutorials. Retrieved November 9, 2026, from https://docs.oracle.com/javase/tutorial/getStarted/intro/definition.html

Oracle. (n.d.-b). *Oracle Java SE support roadmap*. Retrieved November 9, 2026, from https://www.oracle.com/java/technologies/java-se-support-roadmap.html

All diagrams and program listings are original. Explanations are written in original language; standard content on algorithms, languages, translators and object-oriented programming follows Knuth (1997), Deitel and Deitel (2018) and the cited language documentation. Farm values in the examples are illustrative.

Practice MCQs

Q1. A finite, step-by-step procedure for solving a problem is called

(a) a flowchart (b) an algorithm
(c) a compiler (d) a variable

Q2. In a flowchart, a decision is shown by a

(a) rectangle (b) parallelogram
(c) diamond (d) oval

Q3. Which translator converts the whole program into machine code before it runs?

(a) Interpreter (b) Compiler
(c) Debugger (d) Editor

Q4. A program that runs but gives a wrong answer contains a

(a) syntax error (b) logic error
(c) compilation error (d) hardware error

Q5. Which is NOT one of the three basic control structures?

(a) Sequence (b) Selection
(c) Iteration (d) Compilation

Q6. In Visual Basic, the text typed into a TextBox is held in its

(a) Name property (b) Text property
(c) Click event (d) Clear method

Q7. Code that runs when the user clicks a button is an example of

(a) event-driven programming (b) machine language
(c) interpretation (d) normalisation

Q8. In Visual Basic, a variable is declared with the keyword

(a) Var (b) Dim
(c) Let (d) Def

Q9. Java source code is compiled into

(a) machine code for one processor (b) bytecode
(c) HTML (d) assembly language

Q10. Java bytecode is executed by the

(a) JDK compiler
(b) Java Virtual Machine
(c) web browser only
(d) operating system kernel

Q11. Keeping data and methods together and hiding the data inside a class is called

(a) inheritance
(b) polymorphism
(c) encapsulation
(d) iteration

Q12. Which of these is a long-term-support (LTS) release of Java?

(a) Java 22
(b) Java 23
(c) Java 25
(d) Java 26

Answers

Q1 — (b) an algorithm
Q2 — (c) diamond
Q3 — (b) Compiler
Q4 — (b) logic error
Q5 — (d) Compilation
Q6 — (b) Text property
Q7 — (a) event-driven programming
Q8 — (b) Dim
Q9 — (b) bytecode
Q10 — (b) Java Virtual Machine
Q11 — (c) encapsulation
Q12 — (c) Java 25